Task 3 - Calculating Offset Data

Task 3 describes a central part of the project. In this task your goal is to determine an ideal approach trajectory with given parameters and then calculate its distance to the airplane's current position. The results of this calculation will be your **offsets**.

IMPORTANT: This task is completely independent of Tasks 0–2. It runs entirely on arbitrary numbers and values that you define for testing purposes.

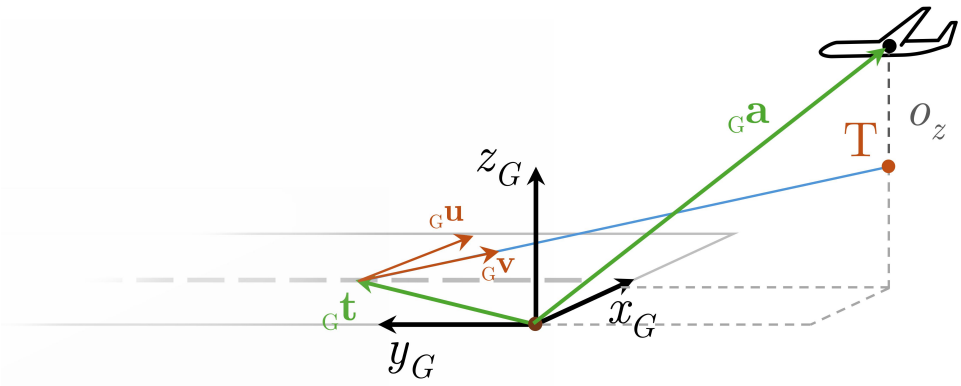
Overall, Task 3 will involve:

1. Defining a linear graph representing the ideal approach trajectory
2. Determining the offsets of the airplane's position relative to the ideal approach trajectory

While working on this task you will assume **four placeholder runway edge coordinates** and **one placeholder airplane coordinate**. Additionally, two variable parameters must be included in your calculations:

- Approach angle (`appAngle`) — The angle at which the aircraft descends during landing and makes contact with the runway. This value describes how steep or shallow the landing approach is.
- Touchdown distance (`tdwnDistance`) — The distance from the start (threshold) of the runway to the exact point where the aircraft first touches down. This value indicates how far along the runway the landing occurs and reflects landing precision.

These placeholders should be defined in your main directly after `pose_transform()`. The coordinates will **later** be replaced by the transformed ArUco marker positions.



With these parameters, you will define a linear function representing the airplane's ideal approach trajectory. The function should take the global coordinates array, `appAngle`, and `tdwnDistance` as input. The output should be the ideal airplane position depending on the airplane's `y_G` coordinate:

- ideal horizontal (`x_G`)
- ideal vertical (`z_G`)

The function should follow this structure:

```
ideal_position[coords_ideal_horizontal, coords_ideal_vertikal] =
    ideal_approach(global_coordinates[], appAngle, tdwnDistance);
```

This function will be used inside the main offset calculation:

```
offsets[offset_horizontal, offset_vertikal] =
    calculate_offsets(global_coordinates[], appAngle, tdwnDistance);
```

At this stage, your main function should include:

- `outline_detect();`
- `pose_detect();`
- `pose_transform();`
- `plotter();`
- `calculate_offsets();`

For this task your solution shall require:

- The beforementioned **main.cpp contents**
- A **function called calculate_offsets**
- A **subfunction called ideal_approach**
- A **well-integrated collection of placeholder coordinates**
- A **detailed wiki** describing the mathematical background of your calculations